

Student Full Name: Seiitkhan Zhannur
 Student Identification Number: 230109009
 Date & Laboratory Session: 17.09.2026
 Workstation / Laptop Make & Model: HP victus
 Processor SKU / SoC Model 12th Gen Intel(R) Core(TM) i5-12450H (2.00 GHz)
 Core Topology (Physical P-cores / E-cores) Physical Cores: _____ | Logical SMT Threads: _____
 Cache Hierarchy Capacity L1: _____ KB | L2: _____ MB | L3: _____ MB
 Host OS & Python Runtime e.g., Linux Kernel 6.8 / Ubuntu 24.04 LTS / CPython 3.11.8

Task 1

Workers p	Measured T(p)	Observed Speedup	Parallel Efficiency
1	3.7945 s	1.00×	100.0%
2	1.5167 s	2.50×	125.0%
4	1.0275 s	3.69×	92.3%
8	0.8748 s	4.34×	54.3%
12	0.8771 s	4.33×	36.1%
16	0.9809 s	3.87×	24.2%

[Q1.1] Identify the exact inflection point (core count p^*) where your observed speedup curve flattens or degrades. Correlate this directly to your machine's physical hardware topology (distinguish between physical Performance cores, Efficiency cores, and logical SMT hyperthreads).

The observed speedup increases rapidly up to 8 worker processes, reaching 4.34×. After this point, scaling flattens: 12 workers provide almost the same speedup (4.33×), while 16 workers reduce the speedup to 3.87×. Therefore, the empirical inflection point is approximately $p^* = 8$ workers.

This indicates that the workload is approaching the available CPU execution resources. Beyond this point, additional worker processes introduce increasing process-management and resource-contention overhead, so adding workers no longer produces proportional speedup.

[Q1.2] Using your empirical measurements for T(1) and T(4), calculate the exact value of P (the parallel fraction) of this workload on your silicon. Show the complete algebraic derivation: $S(4) = T(1)/T(4) = 1 / [(1 - P) + P/4]$.

Using the measured values $T(1) = 3.7945$ s and $T(4) = 1.0275$ s:

$$\begin{aligned}
S(4) &= T(1) / T(4) \\
&= 3.7945 / 1.0275 \\
&\approx 3.6934
\end{aligned}$$

According to Amdahl's Law:

$$S(4) = 1 / [(1 - P) + P/4]$$

Therefore:

$$3.6934 = 1 / [(1 - P) + P/4]$$

$$1 / 3.6934 = 1 - P + 0.25P$$

$$0.27075 = 1 - 0.75P$$

$$0.75P = 0.72925$$

$$P \approx 0.9723$$

Therefore, the measured parallel fraction is approximately 97.23%, while the serial fraction is approximately 2.77%.

[Q1.3] Compute the Asymptotic Speedup Limit $S_{\max} = \lim_{p \rightarrow \infty} S(p) = 1 / (1 - P)$ using your calculated P from Q1.2. Explain what this limit implies if your department purchased a 128-core server node to run this specific workload.

Using the parallel fraction $P \approx 0.9723$:

$$S_{\max} = 1 / (1 - P)$$

$$S_{\max} = 1 / (1 - 0.9723)$$

$$S_{\max} \approx 36.1\times$$

Therefore, the asymptotic speedup limit is approximately 36.1×.

If a 128-core server were used for this workload, the execution would not achieve anything close to 128× speedup. Amdahl's Law predicts that the fixed serial portion of approximately 2.77% limits the maximum theoretical speedup to about 36.1×, even as the number of processors approaches infinity.

Task 2

Memory Layout	Trial 1 [s]	Trial 2 [s]	Trial 3 [s]	Median [s]	Slowdown
Adjacent Indices (False Sharing Active)	1.2872	1.2161	1.3836	1.2872	1.20×
Padded Indices (Cache-Line Aligned)	1.1832	1.0735	1.0500	1.0735	—

[Q2.1] Calculate the exact memory footprint of a 64-bit integer pointer/reference in CPython. Mathematically prove why an index stride of 16 guarantees that adjacent threads never access memory within the same 64-byte L1 cache line.

In CPython on a 64-bit system, a pointer/reference occupies 8 bytes (64 bits). A Python list stores references to Python objects, so each list element occupies an 8-byte pointer slot.

The padded version uses an index stride of 16. Therefore, the memory distance between two adjacent thread positions is:

$$16 \times 8 \text{ bytes} = 128 \text{ bytes}$$

A cache line is 64 bytes. Since:

$$128 \text{ bytes} > 64 \text{ bytes}$$

the memory locations accessed by adjacent threads cannot belong to the same 64-byte cache line. Thus, the stride of 16 provides enough separation to place the accessed list elements on different cache lines.

[Q2.2] Trace the MESI state transitions for the shared cache line containing array elements. When Core 0 executes an increment on index 0 and Core 1 subsequently executes an increment on index 1, describe the bus invalidation messages and state shifts (Shared -> Modified -> Invalid).

Initially, the cache line containing array elements can be in the Shared (S) state when it is present in the caches of multiple cores.

When Core 0 increments index 0, it needs to modify the cache line. The core requests exclusive ownership, changes the line from Shared to Modified (M), and performs the write.

When Core 1 subsequently increments index 1, it accesses the same cache line. Because Core 0 has modified that line, Core 1 cannot modify its copy without first obtaining ownership. A bus coherence request is generated, and the other cached copy is invalidated.

The state transitions can therefore be summarized as:

Shared (S)

- Core 0 obtains write ownership
- Modified (M) on Core 0
- Core 1 requests the same line
- Core 0's copy is invalidated / ownership is transferred
- Invalid (I) for the old copy

This repeated invalidation and cache-line transfer is the main hardware cost associated with false sharing.

[Q2.3] In compiled systems programming, manual array padding wastes memory. Describe two industry-standard compile-time or VM-level mechanisms used to eliminate false sharing (e.g., C++11 'alignas(64)', Java '@Contended', or Go struct field padding)

Two industry-standard mechanisms for reducing false sharing are:

1. C++11 **alignas(64)** — forces a variable or data structure to be aligned to a 64-byte boundary, helping ensure that frequently modified data is placed on separate cache lines.
2. Java **@Contended** — adds padding around fields or objects so that frequently accessed fields are separated onto different cache lines, reducing cache-coherence invalidations between CPU cores.

Another common approach is explicit padding in Go structs, where unused fields are inserted between frequently modified variables so they do not share the same cache line.

Task 3

Counter Architecture	Target Sum	Actual Empirical Sum	Data Corruption Defect Count	Execution Time [s]	Contention Overhead
Unsafe Accumulator (Race Condition)	2,000,000	2,000,000	0	0.1632	1.00x [Baseline]
Locked Accumulator	2,000,000	2,000,000	0 (Protected)	0.4770	2.92x

(threadin g.Lock)					
----------------------	--	--	--	--	--